

Specifications

- Written by Wilson Snyder, with Duane Galbi and Paul Wasson
- Developed on 64-bit Ubuntu
- Should run any system with GCC and Perl
- Works on Redhat Linux, OpenSUSE, HPUX, Solaris, Windows NT under Cygwin (C++ only, no SystemC) and Microsoft Visual C++
- Typically needs a maximum of 1GB for functioning
- GNU lesser-public licence version 3
- Option of using Perl artistic licence
- Latest version 3.900

they make a compelling argument, "Synthesis tools do not comply with Verilog either!"

Verilator takes a cue from here, follows a path closer to the synthesis interpretation and ends up putting together a simulator that gives the same results, but faster. How about when you take the design to the silicon stage? No issue there either, as it is the synthesised netlist that goes into the final wafer.

Optimise and re-optimise

Focus of Verilator lies in optimising your code to the maximum possible extent, while not compromising on performance. It internally creates all states of the resulting simulation and uses a large symbol table for accomplishing its functionality. Thus, each signal access takes about two to three fewer assembly instructions than the usual case, making the execution time 10 to 15 per cent faster. But if there is a change in even one signal, the symbol table needs to be recompiled. Else, it can lead to a lot of complication.

Tightly integrated with C and the like

Just like with Verilog codes, you can make use of C functions the way you wish to. Although there is no support for programming language interface (PLI) as such, common PLI-ish calls like \$display, \$finish or \$time are converted into C++ equivalents. You can even write your own C++ code and integrate it with Verilator to realise

an otherwise complicated piece of functionality. Accessing signals in C just needs a Verilator public comment added, which then allows you to scope into C++ class to read the value of the signal, as you would with any other member variable.

Filling in the gaps

Working with a mix of two languages has its own complications. The easiest, or maybe the most difficult of it all, is finding the right interconnect. If you have a block that just connects cells together, do you code it in Verilog or SystemC? The website of Verilator says, "Everything else being equal, best performance is when Verilator sees all of the design. So, look at the hierarchy of your design, labelling cells as if these are SystemC or Verilog."

Group only SystemCs separately and, similarly, Verilogs separately for achieving best performance. The one place where a pure Verilog might be better off in SystemC category would be if there are multiple instantiations of the same; SystemC might help achieve the result faster! Remember to put any block that uses a combination of the two below SystemC, as Verilator cannot connect to lower-level SystemC cells.

Added support

Verilator makes it easier for you to work with a comprehensive package by extending support to many features that are otherwise limited. In this context, Verilator supports

SystemVerilog Direct Programming Interface (DPI) import and export statements, a limited subset of Verilog Procedural Interface (VPI) and even cross-compiling Verilog code. The latter is often exploited for running the tool on a Linux based system, generating C++ code and then compiling it on Windows. The fact that some parts of System Verilog, Verilog analogue/mixed signal and synthesis directive assertions are also supported, makes the tool popular among regular users. New features are added as and when a user requests it.

See the values changing on the big screen

Verilator provides support for dumping out waveform traces in C++, SystemC or even SystemPerl. You just need to have the necessary packages and point environment variables correctly. For viewing these waveforms on the screen, dump out the standard value change dump (vcd) files and view these with open source tools like Dinotrace or GtkWave. The same will also work with commercially-available tools for waveform viewing, if you so wish.

The website for Verilator, <https://www.veripool.org/wiki/verilator>, has a collection of rather interesting tutorials and presentations that answer doubts about whether this is the right tool for you, how to get high-performance designs and the like. There are also many other applications that are discussed, all of which have used Verilator at some point of time. It even goes on to explain how you could use a tool called vmold to simulate verilated Verilog modules inside MATLAB simulations. Go ahead and take a look for yourself! **EFY**

Priya Ravindran is M.Sc (electronics) from VIT University, Vellore, Tamil Nadu. She loves to explore new avenues and is passionate about writing